M2: Deep Learning for Data Intensive Science

Coursework Assignment

Word Count: 2817

Yilan Xu (yx423)

University of Cambridge

1 Introduction

In this project, we investigate the potential of Large Language Models (LLMs) for time-series prediction. Specifically, we implement the Qwen2.5-Instruct model[1] from HuggingFace and provide it with a prompt consisting of the first 70 time points of a predator-prey system. The model is then tasked with forecasting the subsequent 30 time points, as illustrated in Figure 1.

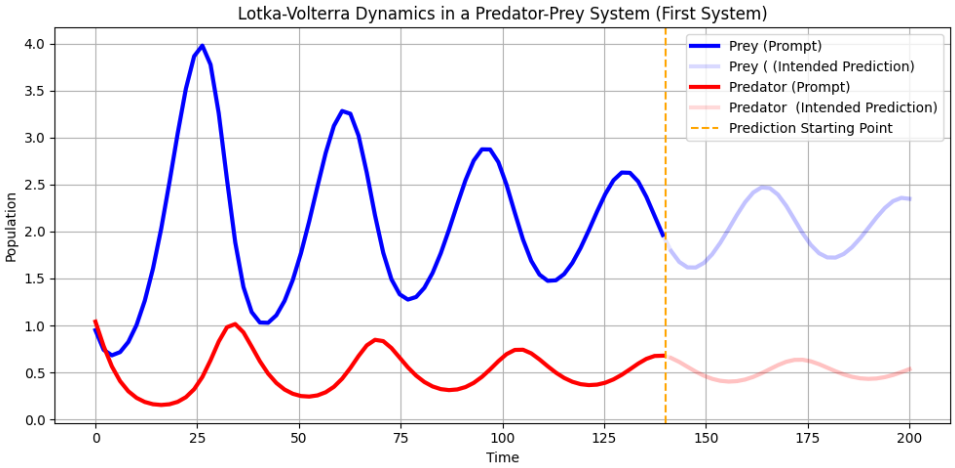

Figure 1: **Lotka-Volterra Dynamics in a Predator-Prey System.** The model is given the first 70% of the time series (up to the orange dashed line), showing the population dynamics of prey (blue) and predator (red). It is then asked to forecast the remaining 30% of the trajectory. Solid lines indicate the prompt given, while faded lines represent the target prediction region. The vertical dashed orange line marks the start of the intended prediction.

To enhance the model's performance, we apply Low-Rank Adaptation (LoRA) for fine-tuning. A strict floating point operations (FLOPs) budget of 10^{17} is enforced consistently across all experiments throughout the project.

2 Baseline

a LLMTIME Preprocessing Scheme

According to Gruver et al.[2], the following data preprocessing steps were performed to standardise input formatting, maximise Qwen's forecasting capabilities, and ensure reliable training and evaluation. All functions are implemented in `src/preprocessor.py`.

a.1 Prepare Dataset

The dataset used in this project was originally shaped as (1000, 100, 2), representing 1000 predator-prey systems, each consisting of one predator and one prey trajectory across 100 time points. The full dataset was split into training and validation sets using an 80:20 ratio. A fixed random seed was applied during the split to ensure reproducibility. The statistical distributions of both the training and validation sets were compared to confirm consistency (Figure 2).

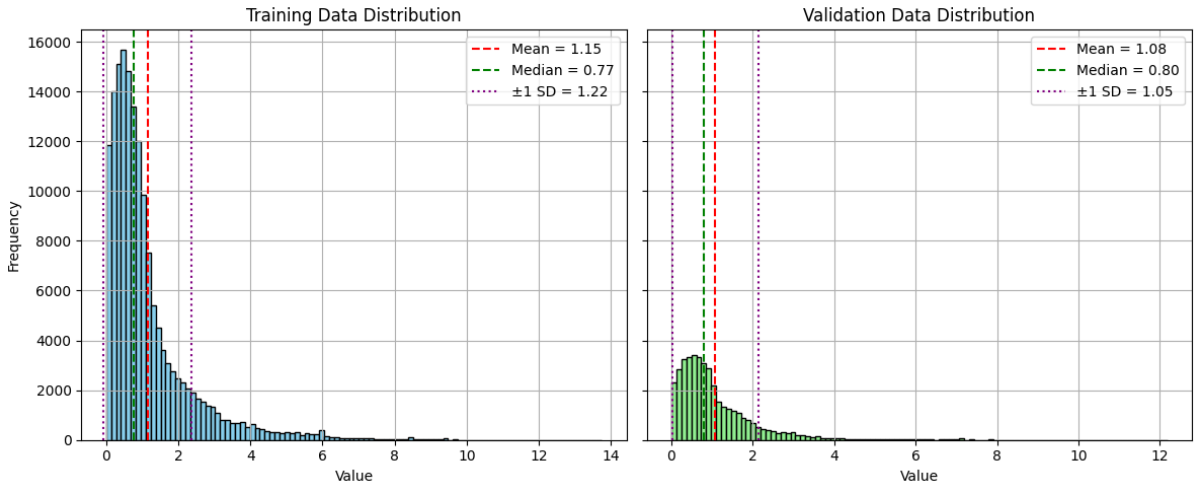

Figure 2: Distribution of values in the training and validation datasets. Both datasets exhibit similar distributions, confirming consistency after splitting.

All data points in the training set were used for training purposes (i.e., 800 systems with 200 data points each). The validation set was further split into two parts: the first part served as input prompts (70 time points for both prey and predator), while the second part represented the ground truth for the expected output predictions (30 time points for both). The prompting portion of the validation set was used both as input for model evaluation and to monitor validation loss during training. The expected predictions were never exposed to the model at any stage, ensuring an unbiased evaluation after training.

a.2 Scaling and Precision

We examined the range and distribution of the training set to determine an appropriate scaling and rounding scheme for the entire dataset. This step ensures a standardised numerical range and consistent token length. Specifically, each data point was scaled using the formula:

$$x_{\text{scaled}} = \frac{x_{\text{original}}}{\alpha}$$

and then rounded to three decimal places.

We selected $\alpha = 0.36$ so that approximately 95% of the values fall within the range $[0, 10]$ (Figure 3). Therefore, most data points would consist of four digits, with some extending to five. By allowing 5% of the data to fall outside this range, the model retains the ability to make predictions beyond the standard token length, thereby supporting more generalised performance.

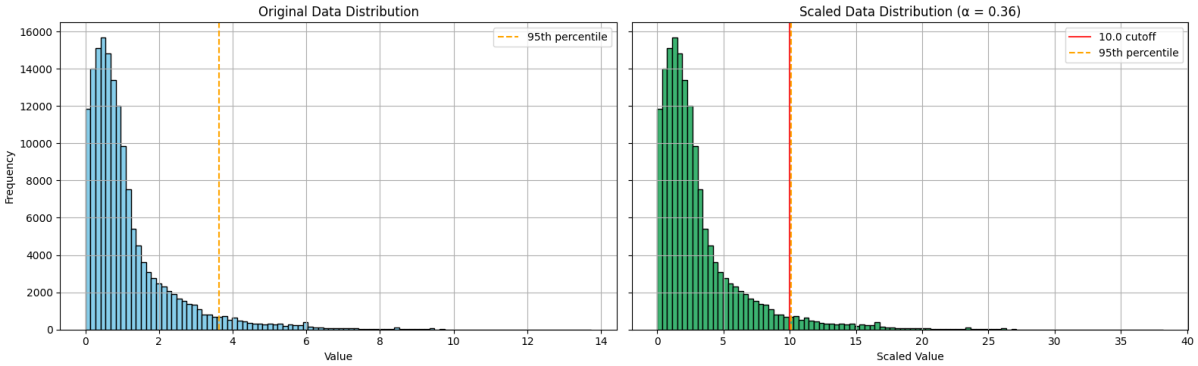

Figure 3: Histogram of original and scaled data distributions. The left panel shows the distribution of original values, with the 95th percentile marked by an orange dashed line. The right panel shows the same data scaled by $\alpha = 0.36$, aligning the 95th percentile with the cutoff value of 10.0 (red solid line).

a.3 Multivariate Time Series Encoding

Each two-variable time series system is encoded as a single string using the following formatting rules:

- A comma (",") separates the prey and predator values at the same time point
- A semicolon (";") separates different time points

a.4 Numeric Tokenisation

Numeric tokenisation is performed using the existing tokenizer from Qwen, which converts each individual numeric or punctuation element into separate tokens (see Table 1).

Table 1: Qwen's String to Token Mapping

Token	','	'.'	'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'	';'
ID	[11]	[13]	[15]	[16]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]	[26]

a.5 Decoding and Inference

To evaluate model predictions, the function `decode_prediction_to_array` is defined to convert output tokens from each system back into numerical arrays with the original dataset shape, i.e., $(100, 2)$.

a.6 Examples

Two example sequences from the dataset are shown in Tables 2 and 3, illustrating the raw values, their scaled and rounded forms, and the final tokenised representations.

Table 2: Example of Data Encoding and Tokenization Process: First three time points from system 811

Original Data	`[[0.92468953, 1.0581826],`
	`[0.6988672, 0.86723346],`
	`[0.60892767, 0.6654375]]`

Scaled, Rounded, Encoded Data	`2.569,2.939;1.941,2.409;1.691,1.848;`
Tokenized Data	`[17, 13, 20, 21, 24, 11, 17, 13, 24, 18, 24, 26, 16, 13, 24, 19, 16, 11, 17, 13, 19, 15, 24, 26, 16, 13, 21, 24, 16, 11, 16, 13, 23, 19, 23, 26]`

Table 3: Example of Data Encoding and Tokenization Process: First three time points from system 549

Original Data	`[[0.8601669, 1.1137589],`
	`[0.4966443, 0.8798939],`
	`[0.35568136, 0.6257832]]`

Scaled, Rounded, Encoded Data	`2.389,3.094;1.380,2.444;0.988,1.738;`
Tokenized Data	`[17, 13, 18, 23, 24, 11, 18, 13, 15, 24, 19, 26, 16, 13, 18, 23, 15, 11, 17, 13, 19, 19, 19, 26, 15, 13, 24, 23, 23, 11, 16, 13, 22, 18, 23, 26]`

The preprocessing scheme is validated as shown in Figure 4.

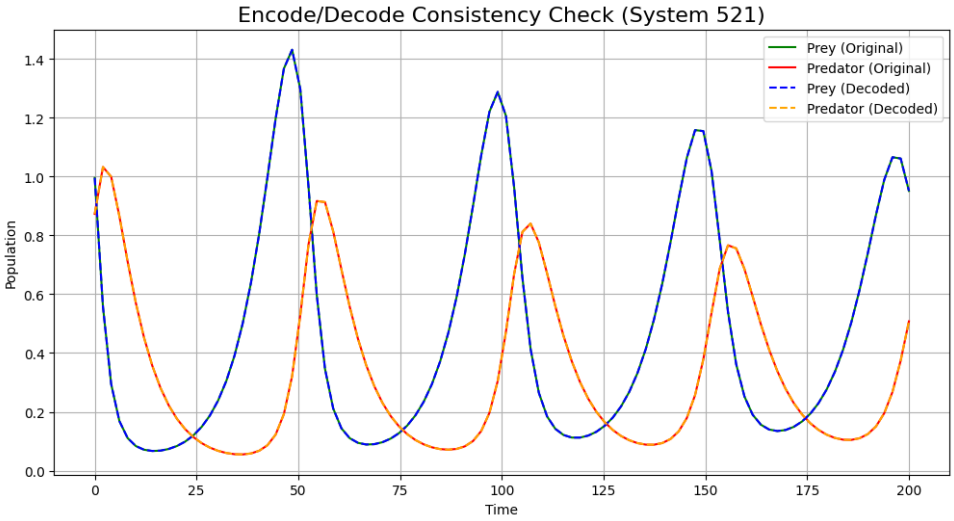

Figure 4: Encode/decode consistency check for System 521. The original prey and predator trajectories are shown alongside the corresponding decoded outputs. The decoded curves closely match the original, demonstrating that the tokenisation and decoding process preserves numerical accuracy throughout the sequence.

b Qwen2.5 Untrained Performance

Evaluation of the untrained model was conducted on the entire tokenised validation set. Inference was performed using deterministic decoding, i.e., by selecting the token with the highest probability at each step (`do_sample = False`).

In this study, the dataset consists of predator-prey systems governed by deterministic mathematical dynamics (specifically, the Lotka–Volterra equations), meaning there is no stochasticity to learn and a unique correct value exists for each time point. Therefore, our goal is to achieve the highest possible prediction accuracy, and we focus solely on the most likely output sequence. This setup ensures stable outputs and enables reliable evaluation.

For the evaluation metric, mean absolute error (MAE) was primarily used to quantify the model's average prediction accuracy. MAE is relatively robust to outliers—often present in LLM-generated predictions due to their autoregressive nature—and thus better reflects the overall performance, which is the main focus of this study.
Root mean square error (RMSE) was also reported to provide complementary information, as it is more sensitive to large deviations and therefore highlights the impact of outliers.

To measure MAE, the prompting portion of the output sequence was removed, and the predictions (i.e., time points 70–100) were aligned with the corresponding ground truth. MAE was then computed for each system individually. The average MAE across all 200 validation systems (Figure 5), for the untrained `Qwen2.5-Instruct` model, was 0.86 ± 1.62. The distribution is skewed towards smaller values, with the average MAE approximately 80% of the ground truth mean. This suggests that the model's prediction errors are on the same order of magnitude as the target values themselves, which indicates a modest ability to follow general trends, though far from accurate modelling.

Table 4: Untrained Qwen2.5-Instruct model's forecasting ability

Metric	**Value**
Ground Truth Mean Prediction	1.10
Mean Absolute Error (MAE)	0.86 ± 1.62
Root Mean Squared Error (RMSE)	2.95 ± 2.68

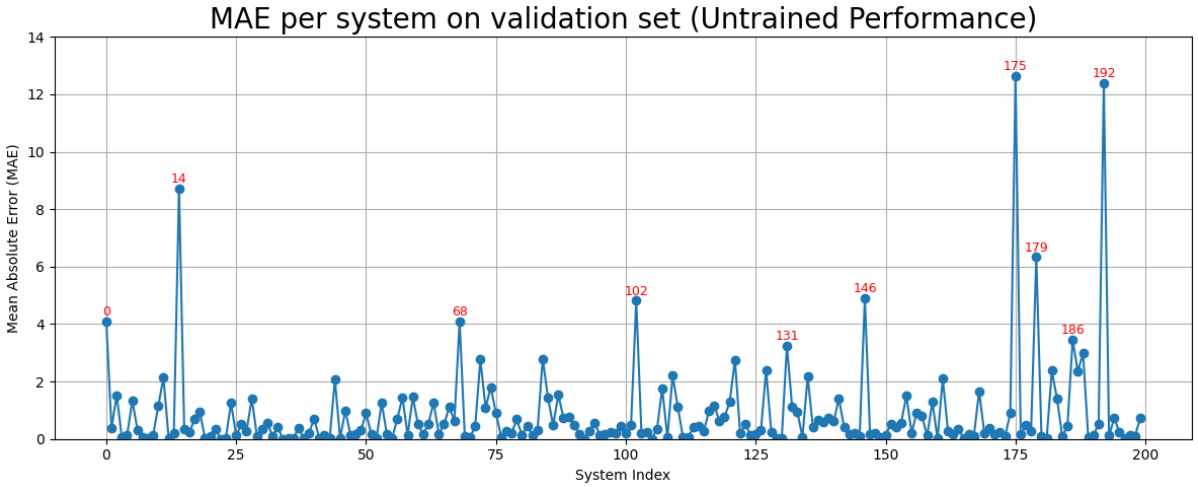

Figure 5: MAE for each of the 200 validation systems using the untrained model. Systems with MAE greater than 3 are highlighted in red, indicating failed predictions.

Notably, 95% of the MAE values lie within the range $[0, 3]$, implying that the untrained Qwen model is already capable of capturing the basic trajectory patterns in most systems (Figure 6). However, several outlier cases with MAE greater than 5 exhibit completely failed predictions. This is also reflected in the high variance of MAE and the large root mean square error (RMSE = 2.95), both of which indicate severe output instability in certain scenarios.

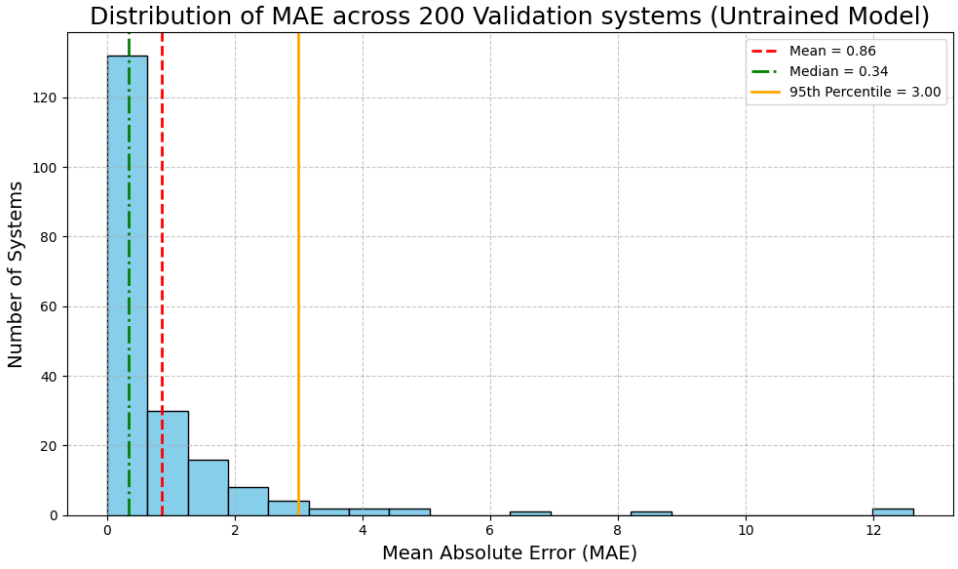

Figure 6: Distribution of mean absolute error (MAE) across 200 validation systems for the untrained model. The red dashed line indicates the mean MAE (0.86), the green dash-dotted line shows the median (0.34), and the orange solid line marks the 95th percentile (3.00).

Three systems with good forecasting performance (MAE < 0.1) and three systems with the highest MAE values are shown in Figure 7 as representative good and bad examples.

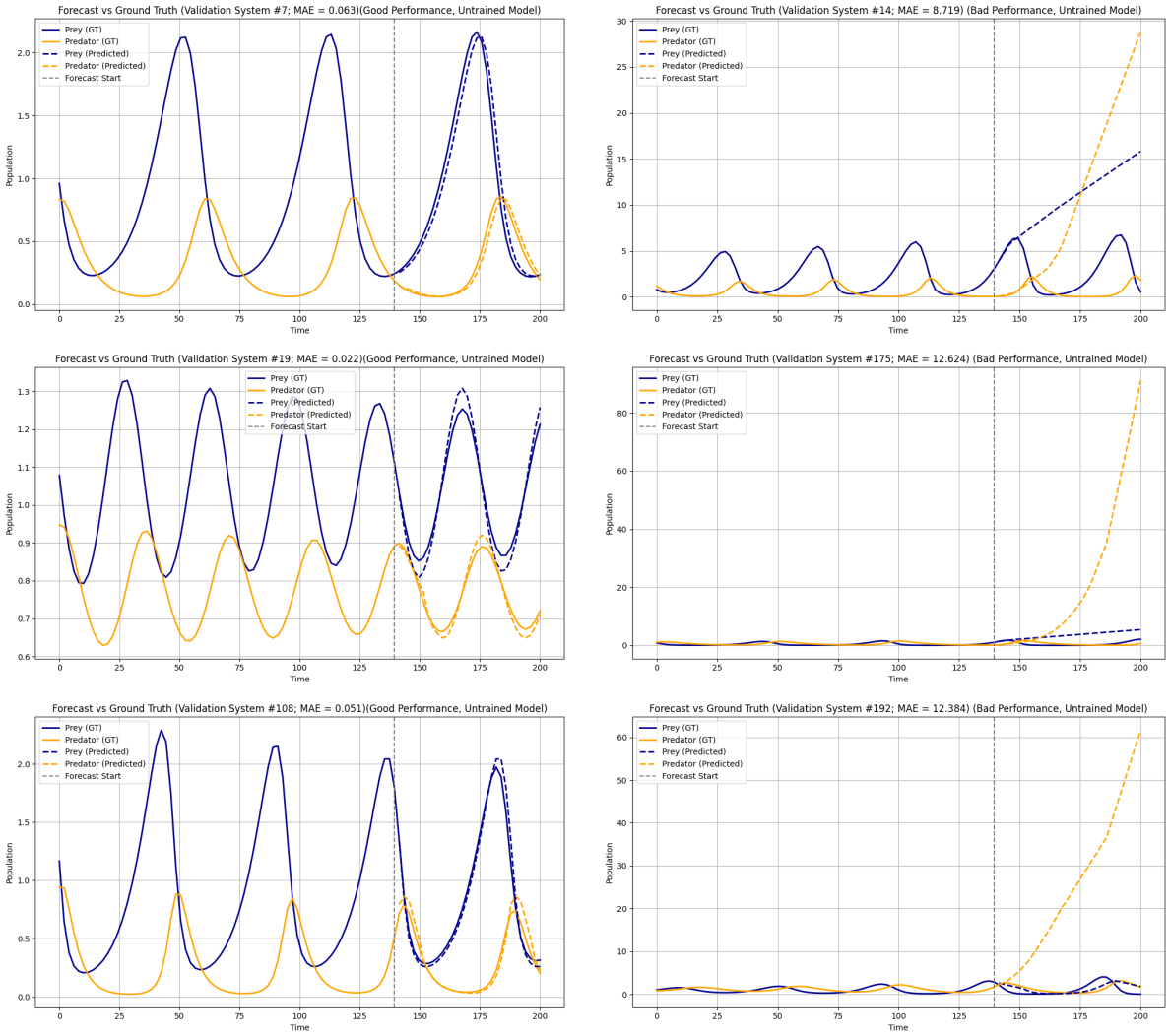

Figure 7: Examples of prediction performance from the untrained model on selected validation systems. Left column: systems with good forecasting performance (MAE<0.1). Right column: systems with poor forecasting performance (high MAE). Solid lines represent ground truth, dashed lines show model predictions, and the vertical grey dashed line indicates the start of the prediction window.

c Compute Constraint

This study is constrained by a maximum budget of 10^{17} floating point operations (FLOPs) across all training experiments. FLOPs are estimated based on the input size and model hyperparameters, and are accumulated layer by layer. To compute the total FLOPs for Qwen2.5, we followed its transformer architecture and configuration (Figure 8), and decomposed each module into elementary operations.

```
Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=True)
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): Linear(in_features=896, out_features=128, bias=True)
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
        (mlp): Qwen2MLP(
          (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
          (up_proj): Linear(in_features=896, out_features=4864, bias=False)
          (down_proj): Linear(in_features=4864, out_features=896, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
        (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      )
    )
    (norm): Qwen2RMSNorm((896,), eps=1e-06)
    (rotary_emb): Qwen2RotaryEmbedding()
  )
  (lm_head): Linear(in_features=896, out_features=151936, bias=True)
)
```

Figure 8: Baseline architecture of the Qwen2.5 model used in FLOP analysis.

Table 5 provides a summary of the key operations contributing to the FLOP count in each component of the Qwen2.5 model. We apply certain simplication compared to the actual model (e.g., ignored Grouped Query Attention), and each operation is mapped to the corresponding FLOP cost following the given FLOPS count scheme. For a single forward pass with batch size = 1, sequence length = 512, and all other default parameters of Qwen2.5, the total FLOPs without LoRA applied is approximately 5.1×10^{11}.

Table 5: Operation-wise FLOPs Accounting in the Transformer Architecture.

Component	**Operation**	**FLOPs Function Input**	**#**
Positional Encoding	$X^{D\times N} \leftarrow X^{D\times N} + p^{D\times N}$	Embedding Matrix shape (D, N), D is embedding dim (896) and N is context length	1
Input LayerNorm	$X^{D\times N} \leftarrow \text{RMSNorm}(x) = \frac{x}{\sqrt{\|x\|_2^2/D+\epsilon}}$	(D, N)	24
Self Attention (Multi-head)	Q_i, K_i, V_i: $3h \times \left(W^{\frac{D}{h}\times D} \cdot X^{D\times N} + \beta^{\frac{D}{h}\times N}\right)$ Attention (A_i): $h \times \text{Softmax}(K_i^\top Q_i)$ Head Output: $h \times V_i \cdot A_i$ $X^{D\times N} \leftarrow W^{D\times D} \cdot \text{Concat}(\text{Head Output}_i)$ Residual: $X^{D\times N} \leftarrow X^{D\times N} + \text{Residual}$	(D, N) h: # heads (14)	24
Post-Attention LayerNorm	$X^{D\times N} \leftarrow \text{RMSNorm}(x) = \frac{x}{\sqrt{\|x\|_2^2/D+\epsilon}}$	(D, N)	24
MLP with SwiGLU	$\text{up} = W_u^{H\times D} \cdot X^{D\times N}$ $\text{gate} = W_g^{H\times D} \cdot X^{D\times N}$ $\text{activation} = \text{SiLU}(\text{gate})$ $\text{hidden} = \text{activation} \odot \text{up} \quad (\in R^{H\times N})$ $X^{D\times N} \leftarrow \text{down} = W_d^{D\times H} \cdot \text{hidden}$ Residual: $X^{D\times N} \leftarrow X^{D\times N} + \text{Residual}$	(D, N) H: MLP hidden dim (4864)	24
Output LayerNorm	$X^{D\times N} \leftarrow \text{RMSNorm}(x) = \frac{x}{\sqrt{\|x\|_2^2/D+\epsilon}}$	(D, N)	1
LM Head (Output Projection)	$X^{O\times N} \leftarrow W^{O\times D} \cdot X^{D\times N}$	(D, N) O: output dimension (151936)	1

Table 6 presents the additional FLOP count introduced by applying LoRA (Low-Rank Adaptation) to the query and value projection layers in the self-attention mechanism. For a single forward pass with batch size = 1, sequence length = 512, rank = 4, and all other parameters fixed according to the Qwen2.5 configuration, LoRA introduces approximately 4.0×10^8 additional FLOPs.

This added computational cost is negligible compared to the overall FLOPs of the model, demonstrating that LoRA contributes only a small number of operations relative to the full architecture.

Table 6: FLOPs Accounting for LoRA (Low-Rank Adaptation)

Component	**Operation**	**FLOPs Function Input**	**#**
LoRA (Low-Rank Adaptation)	Low-rank matrices: $A \in R^{r\times \frac{D}{h}}$, $B \in R^{\frac{D}{h}\times r}$ Projection: $2h \times [P = B(AX)]$ Output: $2h \times [X \leftarrow X + \frac{\alpha}{r} \cdot P]$	(D, N) r: rank h: # heads (14)	24

To compute the total FLOP count for a complete training experiment, the FLOPs required for a single forward pass are multiplied by the batch size and the number of training steps. An additional factor of 3 is applied to account for both the forward and backward passes (for backpropagation).

In summary, the FLOP estimation for any experiment depends on four input variables: batch size, number of steps, context length, and the LoRA rank (if applicable). All other architectural parameters—such as embedding dimension, number of attention heads, and hidden layer size in the MLP—are fixed as defined in the Qwen2.5 model configuration.

3 LoRA Implementation

LoRA (Low-Rank Adaptation) is employed in Qwen2.5 to fine-tune its ability for time-series prediction. LoRA functions by freezing the original pretrained weights W in the large model and introducing an additional trainable update through low-rank matrices A and B. These matrices are inserted into specific linear layers—namely, `q_proj` and `v_proj` within the self-attention module. This approach enables efficient task-specific adaptation with significantly fewer trainable parameters, as illustrated below:

Original output:	$y_{\text{base}} = Wx \quad \text{where } W \in R^{d_{\text{out}} \times d_{\text{in}}}$
LoRA trainable path:	$y_{\text{LoRA}} = BAx \quad \text{where } A \in R^{r \times d_{\text{in}}},\ B \in R^{d_{\text{out}} \times r}$
Final output:	$y = y_{\text{base}} + \frac{\alpha}{r} \cdot y_{\text{LoRA}} = Wx + \frac{\alpha}{r} \cdot (BAx)$

where α is a scaling factor (set to r in this project), and r is the rank of the low-rank decomposition.

In this project, only 422,272 out of a total of 494,455,040 parameters are trainable, with the trainable subset introduced solely by the LoRA modules.

```
Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): LoRALinear(
            (original_linear): Linear(in_features=896, out_features=896, bias=True)
          )
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): LoRALinear(
            (original_linear): Linear(in_features=896, out_features=128, bias=True)
          )
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
        (mlp): Qwen2MLP(
          (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
          (up_proj): Linear(in_features=896, out_features=4864, bias=False)
          (down_proj): Linear(in_features=4864, out_features=896, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
        (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      )
    )
    (norm): Qwen2RMSNorm((896,), eps=1e-06)
    (rotary_emb): Qwen2RotaryEmbedding()
  )
  (lm_head): Linear(in_features=896, out_features=151936, bias=True)
)
```

Figure 9: Model architecture of Qwen2.5 with LoRA layers. Each attention sub-module includes a LoRALinear wrapper around the original linear projections for q and v, enabling low-rank adaptation during fine-tuning. Other structural components such as RMSNorm, MLP blocks, and rotary embeddings are retained as in the base Qwen architecture.

a Train with default LoRA

a.1 Training Setup

All training procedures described in the following sections were carried out on the tokenised training set introduced in Section 2.a.1. Each sequence of tokens was further divided into overlapping chunks based on the specified context length. Training was performed in batches, with each batch consisting of 4 such chunks. The training loss was periodically monitored by averaging the batch-wise loss over each evaluation interval.

For validation, the prompting segments of the validation set (i.e., the first 70 time points of each system) were used to compute the validation loss during training. Since these segments do not include the ground truth values that the model is expected to predict during inference, this setup prevents data leakage. Validation loss was tracked at the same intervals as training loss and was computed across the entire validation set for each experiment.

a.2 Overfitting Test

Before proceeding with full-scale training for time-series prediction, a sanity check was conducted to ensure that both the model and training pipeline were functioning as expected. In this test, the LoRA-augmented model was trained on a small subset comprising 0.5% of the total training data, using a relatively high learning rate (3×10^{-4}) and default hyperparameters for 1000 steps. As anticipated, the model successfully overfitted this small dataset (Figure 10), which confirmed the correct implementation of the model and training setup.

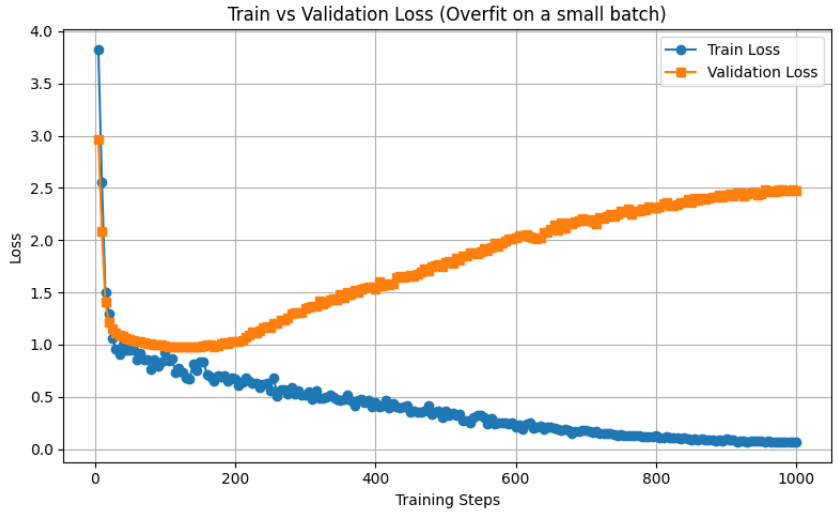

Figure 10: Training and validation loss over 1000 steps during the overfitting test on a small subset (0.5%) of the training data. The training loss steadily decreases towards zero, while the validation loss increases after an initial drop, indicating successful overfitting and verifying the correct functioning of the model and training pipeline.

a.3 Default LoRA Performance

- **Training**
  In this experiment, training was conducted on the full training set (800 systems). The model was trained using a single LoRA configuration with default hyperparameters: rank = 4, learning rate = 1×10^{-5}, and maximum context length = 512. The training was performed for 3,000 steps, with training and validation losses recorded every 100 steps (Figure 11).

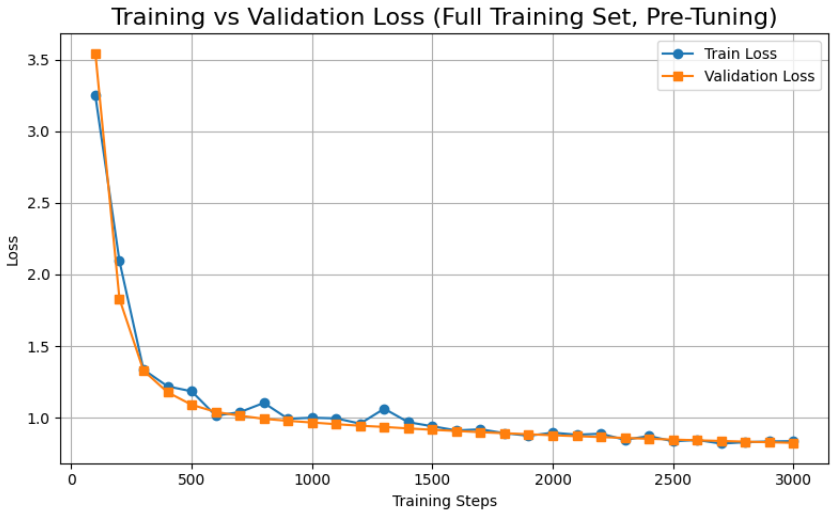

Figure 11: Training and validation loss over 3,000 steps using the full training set with the default LoRA configuration (rank = 4, learning rate = 1×10^{-5}, context length = 512). Both losses decrease steadily and remain closely aligned throughout training, indicating stable convergence without overfitting.

Both training and validation loss curves exhibit a monotonically decreasing trend, indicating that the model learns effectively over time. Initially, both losses drop sharply, reflecting rapid early learning, and the rate of decrease slows as the model approaches convergence. The two curves remain closely aligned throughout the training process, showing no signs of overfitting.

- **Evaluation**
  Evaluation was conducted on the full validation set (200 systems, using the first 70 time points as input prompts). The average MAE across all systems for the `Qwen2.5-Instruct` model after LoRA fine-tuning was 0.78 ± 2.58 (Figure 12). Compared to the untrained model, this represents a modest improvement in mean prediction accuracy, with a reduction of 0.08 in MAE.

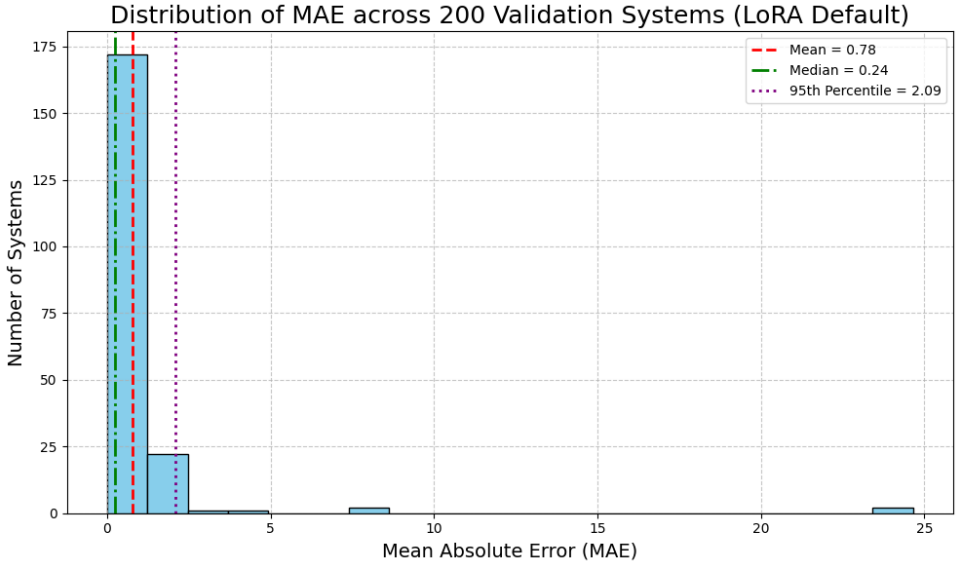

Figure 12: Distribution of mean absolute error (MAE) across 200 validation systems after fine-tuning with default LoRA configuration.

However, the standard deviation of MAE nearly doubled, indicating a higher variance in prediction quality. As shown in the MAE distribution (Figure 13), two systems—#84 and #100—exhibited

extremely high MAE values (Figure 14), where predictions diverged drastically, suggesting instability or failure in certain edge cases.

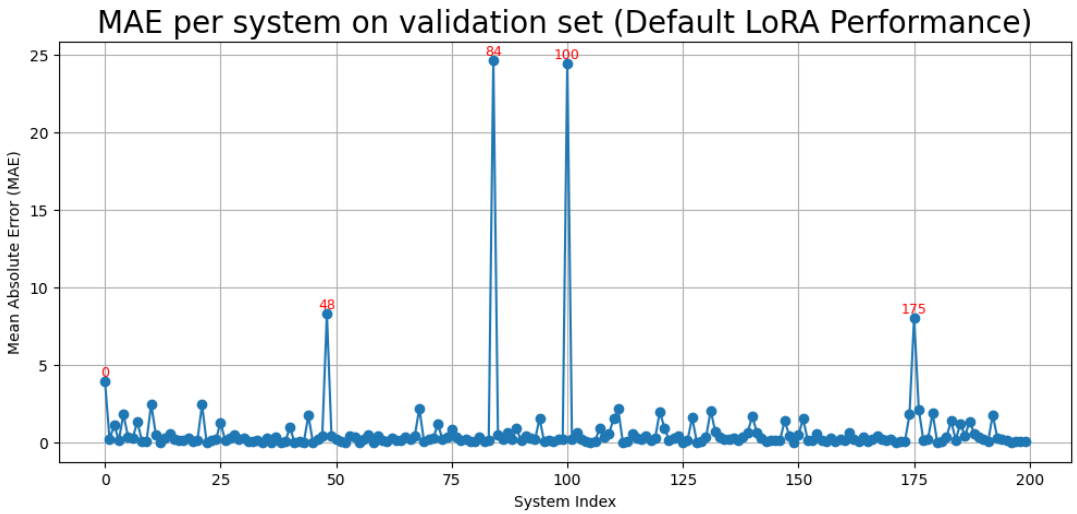

Figure 13: MAE per system on the validation set after LoRA fine-tuning. Most systems exhibit low error, while a few systems (e.g., #84 and #100) have extremely high MAE values. Systems with MAE > 3 are labelled in red to indicate poor performance.

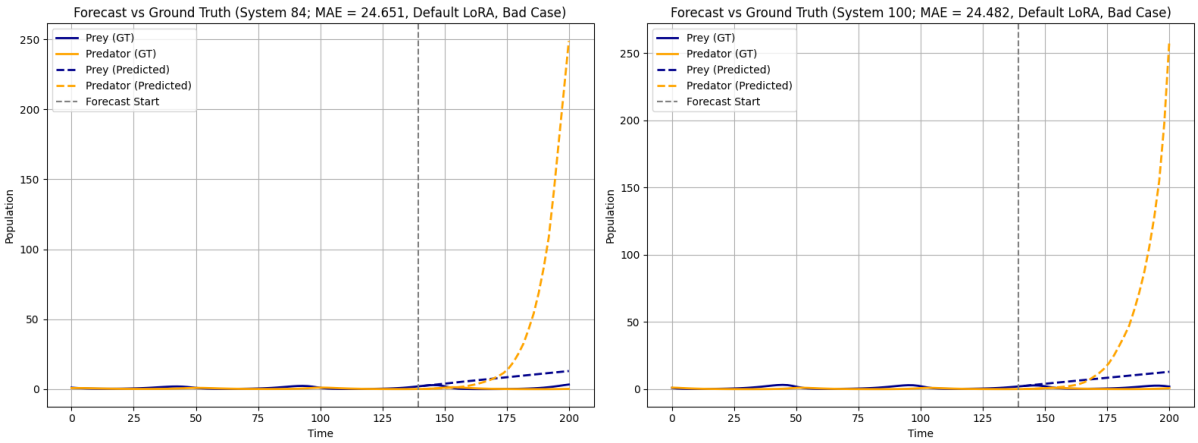

Figure 14: Predictions vs. ground truth for two validation systems (#84 and #100) with the highest MAE values after LoRA fine-tuning.

Despite this, the number of poorly performing systems (defined as MAE > 3) dropped from 10 (in the untrained model) to only 4 after fine-tuning, indicating more reliable forecasting in the majority of cases. Additionally, the 95th percentile MAE was reduced to 2.1, significantly lower than in the untrained baseline. This overall improvement is further reflected in the lower RMSE (1.36 ± 5.71), suggesting that the fine-tuned model produces fewer extreme outliers.

Table 7: Forecasting performance of default LoRA-tuned `Qwen2.5-Instruct` model

Metric	**Value**
Ground Truth Mean Prediction	1.10
Mean Absolute Error (MAE)	0.78 ± 2.58
Root Mean Squared Error (RMSE)	1.36 ± 5.71

We visualise the same six systems from Section 2b for the untrained model as a comparison (Figure 15). Interestingly, all three systems that initially exhibited good performance showed larger prediction errors after training, with more noticeable deviations. Conversely, the three systems that originally performed poorly demonstrated reduced MAE and improved alignment with the ground truth after training, suggesting that the model had learned to capture some underlying structure in the time series.

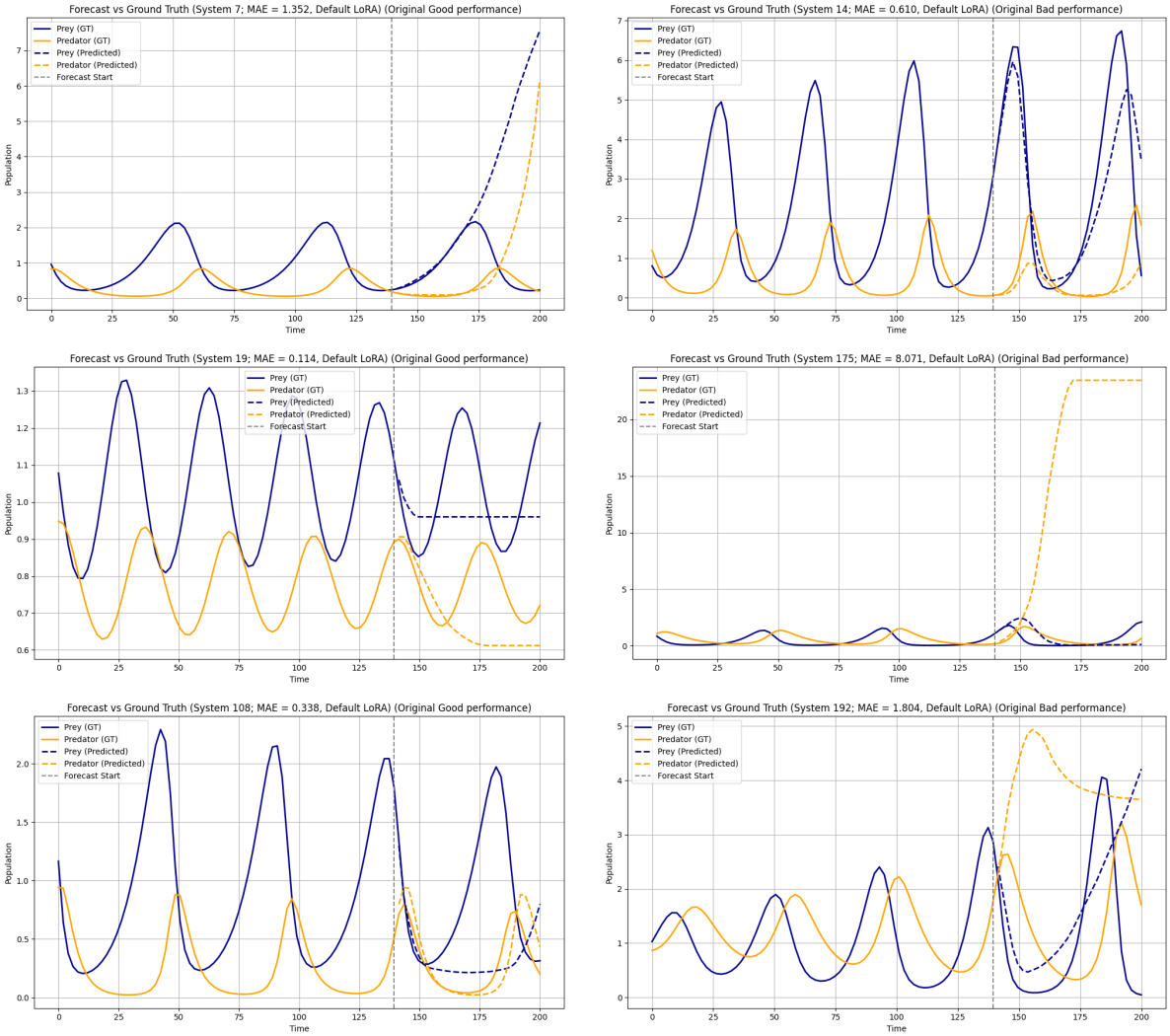

Figure 15: Prediction performance of the default LoRA-tuned model on the same six validation systems shown in Figure 7. Left column: systems that originally had good performance under the untrained model. Right column: systems that originally performed poorly. Default LoRA fine-tuning leads to improved forecasts in previously poor-performing systems, while some degradation is observed in originally well-performing ones. Solid lines represent ground truth; dashed lines show model predictions. The vertical grey dashed line indicates the start of the prediction window.

To summarise, the Qwen 2.5 model with the default LoRA configuration, exhibited modest overall improvement in time-series forecasting by reducing outliers and producing more balanced performance across most systems. It also shows instability in some predictions.

4 Hyperparameter Tuning

Hyperparameter tuning was performed to further improve the model's performance. To ensure computational efficiency, all experiments were conducted at a reduced scale. Specifically, only the first 30% of the training and validation sets were used, corresponding to 240 and 60 systems, respectively. Each experiment was trained for 600 steps to allow fair comparison, and losses were logged every 10 steps. The set of hyperparameters that yielded the lowest validation MAE was selected for subsequent experiments.

- **Grid Search on Learning Rate and LoRA Rank**
  A grid search was conducted over a 3×3 combination of learning rates $(1\times10^{-5}, 5\times10^{-5}, 1\times10^{-4})$ and LoRA ranks $(2, 4, 8)$. The training and validation loss curves for all configurations are shown in Figure 16. All combinations demonstrated a generally decreasing loss trend, and training and validation curves remained closely aligned, with no signs of overfitting. This validates the reliability of the comparison across configurations.

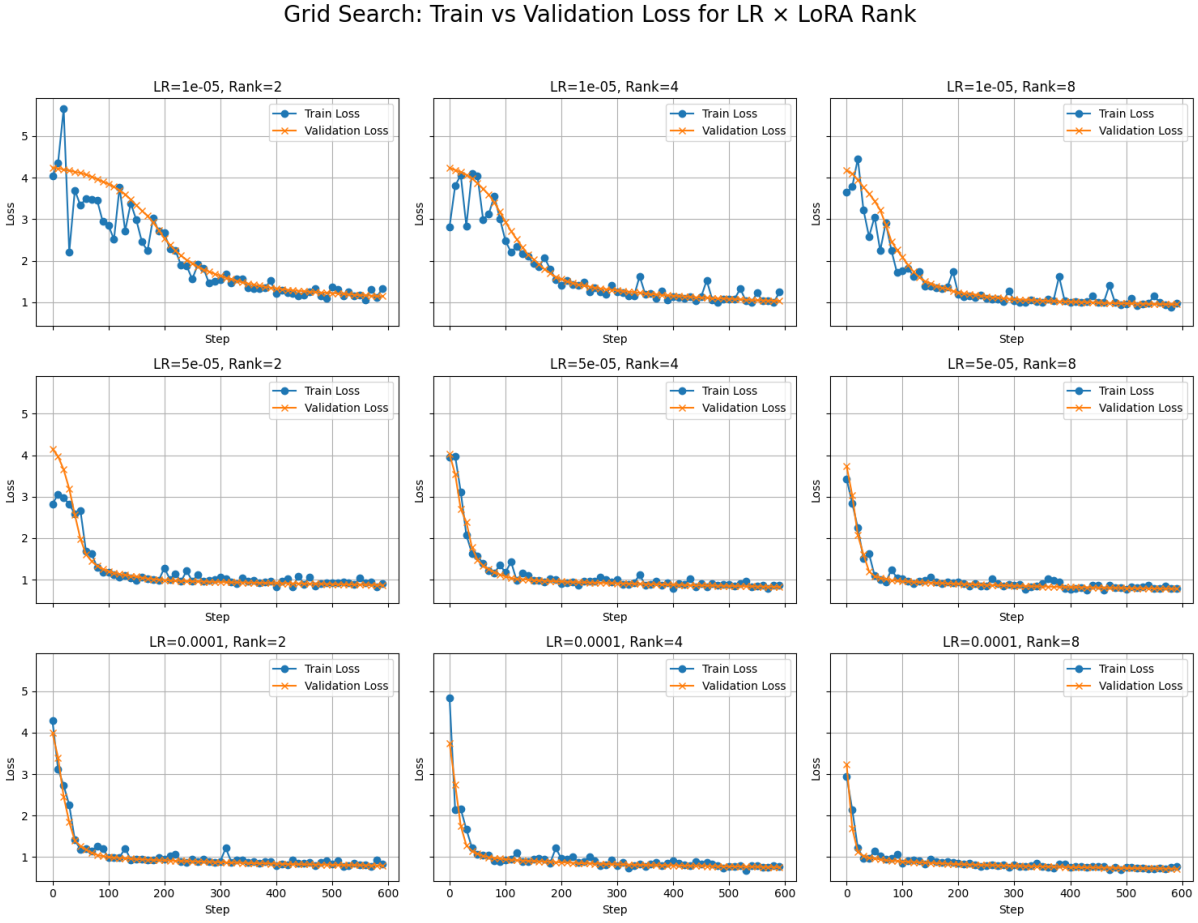

Figure 16: Training and validation loss curves across nine combinations of learning rate and LoRA rank during grid search. Each subplot shows the performance over 600 training steps using a reduced dataset (30% of the full training and validation sets). All combinations exhibit decreasing trends and stable convergence without overfitting, which validates the consistency and comparability of results across hyperparameter settings.

Table 8 summarises the MAE values for all nine combinations. No clear monotonic trend was observed with respect to either hyperparameter. The lowest MAE was achieved with a learning

rate of 1×10^{-4} and a LoRA rank of 8. This configuration was therefore selected for the subsequent experiment.

Table 8: MAE for LoRA Grid Search.

Learning Rate	**Rank = 2**	**Rank = 4**	**Rank = 8**
$\mathbf{1 \times 10^{-5}}$	0.67	0.90	0.55
$\mathbf{5 \times 10^{-5}}$	2.63	1.05	0.36
$\mathbf{1 \times 10^{-4}}$	0.49	0.31	**0.26**

- **Effect of Context Length**
  Additional experiments were conducted to evaluate the impact of different context lengths (128, 512, 768) on model performance. Among these, the model trained with a context length of 512 was already covered in the previous section as part of the default configuration. Therefore, only the additional two experiments (128 and 768) contribute to the total FLOP count.

  Training and validation losses for all three configurations are shown in Figure 17.

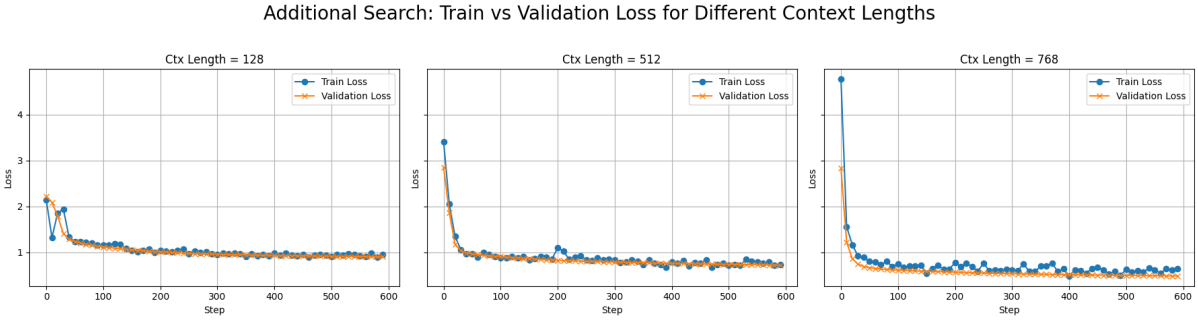

Figure 17: Training and validation loss curves for three different context lengths (128, 512, 768). All models were trained for 600 steps on 30% of the training and validation sets. No overfitting was observed. Increasing the context length leads to faster convergence and lower final losses, indicating improved learning capacity.

Evaluation results from inference are summarised in Table 9, which confirm that a context length of 768 yields the best forecasting performance. Increasing the context length significantly improves prediction accuracy, likely due to the model's enhanced ability to incorporate longer historical dependencies. However, this improvement comes at the cost of largely increased computational overhead.

Table 9: Effect of Context Length on MAE

Context Length	**MAE**
128	0.86
512	0.24
768	**0.22**

5 Final Training

The final training was performed on the full training and validation sets for 3,000 steps, with losses logged every 100 steps. This setup is consistent with the training configuration used in Section 3a. The optimal hyperparameter combination identified from previous experiments—learning rate of 1×10^{-4},

LoRA rank of 8, and context length of 768—was used.

During training, the training loss dropped significantly within the first 500 steps, then gradually decreased at a slower rate before fully converging around step 1,500. This suggests that the model had reached its optimal performance under the chosen hyperparameter configuration (Figure 18). Interestingly, the validation loss remained consistently lower than the training loss throughout, likely because the validation set consists only of the initial prompting segments (i.e., the first 70 time points), which are inherently easier to predict than the full-length training sequences.

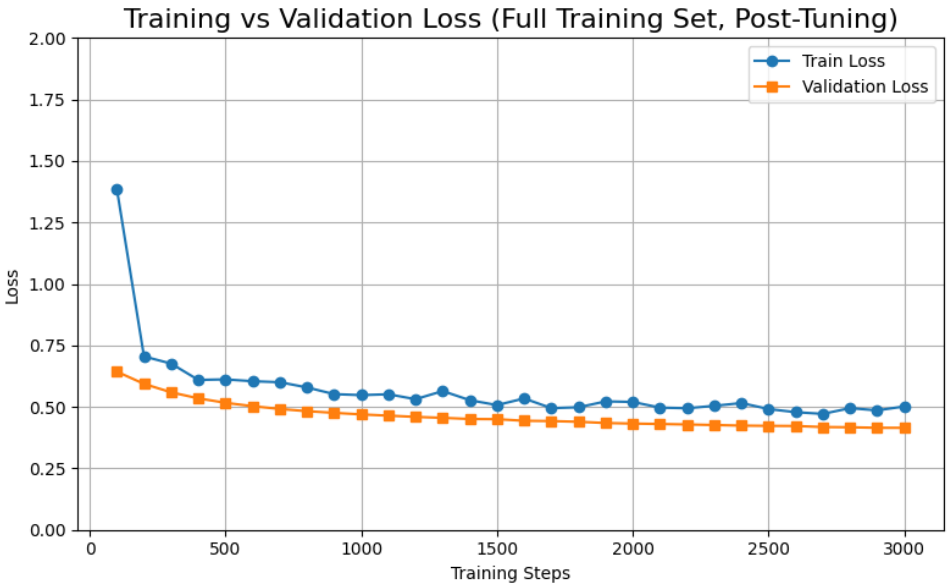

Figure 18: Training and validation loss over 3,000 steps using the full dataset and optimal hyperparameters (learning rate $= 1 \times 10^{-4}$, LoRA rank $= 8$, context length $= 768$). Training loss converges steadily, and validation loss remains consistently low, showing no signs of overfitting.

Evaluation results are presented in Table 10. The fine-tuned LoRA model demonstrated strong forecasting performance, with very low MAE and RMSE, and significantly smaller standard deviations compared to the model trained with default hyperparameters. Notably, 95% of the MAE values lie below 0.57—lower than the average MAE observed in both the untrained model and the default LoRA configuration (Figure 19). This highlights the impact of hyperparameter tuning.

Table 10: LoRA finetuned Qwen2.5-Instruct model's forecasting ability

Metric	**Value**
Ground Truth Mean Prediction	1.10
Mean Absolute Error (MAE)	0.17 ± 0.18
Root Mean Squared Error (RMSE)	0.39 ± 0.41

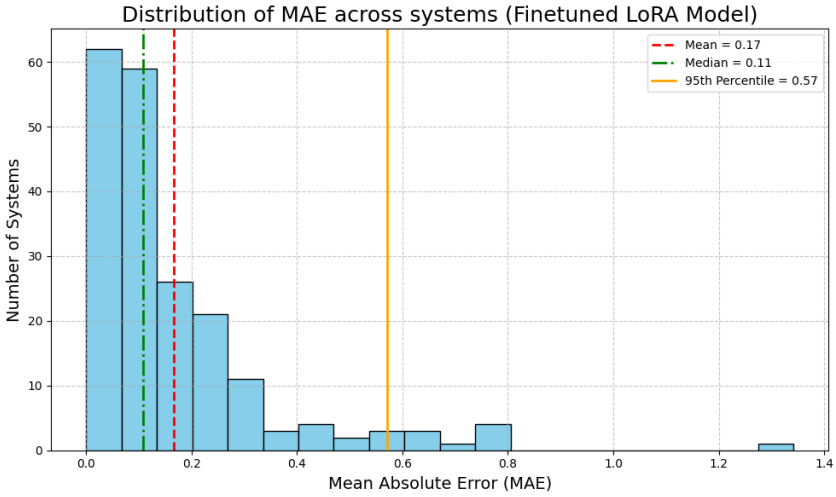

Figure 19: Distribution of MAE across all 200 validation systems using the fine-tuned LoRA model with optimal hyperparameters. Most systems achieve very low MAE, with no extreme outliers observed.

No outliers were observed in the final predictions. Figure 20 highlights the systems that previously showed poor performance under the untrained model (i.e., MAE > 3); all of these now exhibit MAE values below 1, reflecting consistent and substantial improvements.

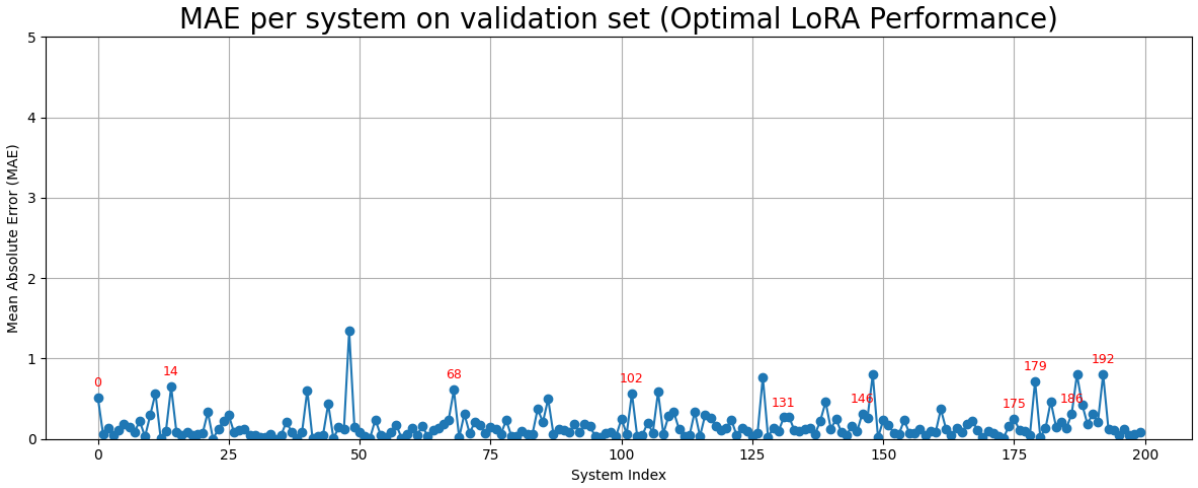

Figure 20: MAE per validation system under the fine-tuned LoRA model. All previously high-error systems (highlighted in red) now have MAE below 1, which indicates consistent improvements of predictions.

The same six systems are visualised once again (Figure 21). In all cases, the model successfully captures the general trend of the time-series data during inference, with a substantial portion of the predictions aligning with the ground truth values.

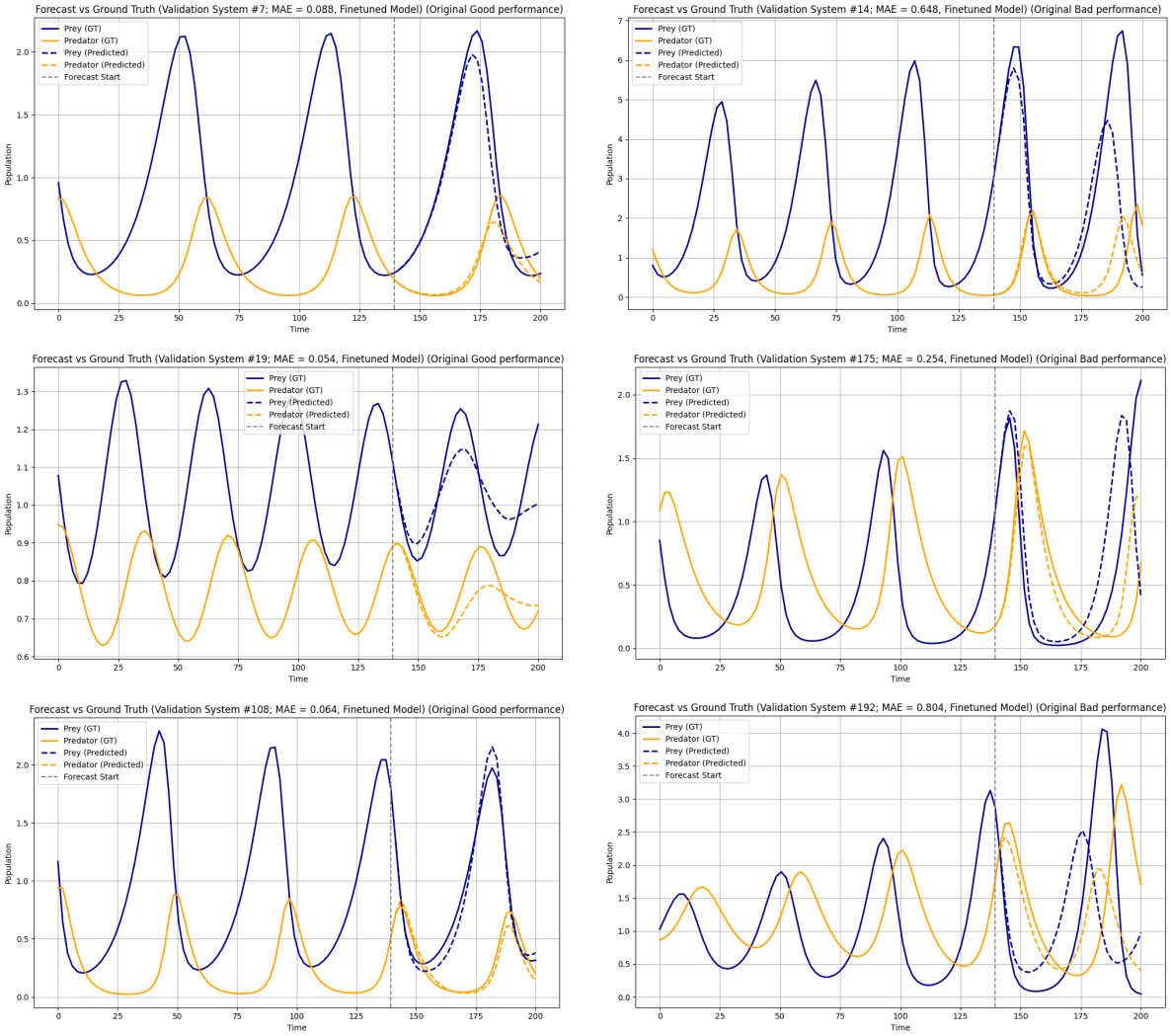

Figure 21: Prediction performance of the fine-tuned LoRA model with optimal hyperparameters, on the same six validation systems shown before. Left column: systems that originally had good performance under the untrained model. Right column: systems that originally performed poorly. The fine-tuned model achieves accurate predictions in both cases, capturing the overall dynamics of the time series. Previously high-error systems now exhibit agreement with ground truth. Solid lines represent ground truth; dashed lines show model predictions. The vertical grey dashed line marks the start of the prediction window.

6 FLOPS Count

Table 11: FLOPs Usage by Experiment Stage

Experiments	**FLOPs (in 10^{17})**
Overfit Test	0.061
3(a) LoRA implemented with default hyperparameters	0.183
3(b) Learning rate & LoRA rank 3×3 grid search	0.330
3(b) Context length tuning	0.065
3(c) Final training	0.282
Total FLOPs Count	**0.921**

7 Conclusion

a Recommendations

- Large language models are well-suited for time-series prediction, as they naturally handle multi-modal distributions and repetitive patterns. They offer a strong starting point compared to training models from scratch.
- Time-series data should be consistently and logically formatted before training (e.g., as structured prompts), which enables models to better generalise patterns.
- For efficient fine-tuning, Low-Rank Adaptation (LoRA) is recommended. By freezing most parameters and only updating small trainable matrices, LoRA enables fast convergence and strong performance under limited compute budgets.
- Given the fast convergence of LoRA fine-tuning, it is important to monitor training and validation loss closely to avoid unnecessary training steps and potential overfitting.
- Hyperparameter tuning is crucial for LoRA to significantly improve prediction accuracy. Use small training sets and a limited number of steps for efficient hyperparameter search, as they are often sufficient to identify effective configurations.
  For similar prediction tasks, a high LoRA rank and learning rate have proven to give the best results with minimal compute overhead. While longer context lengths reduced error, the gains quickly plateaued as FLOPs grew — making moderate lengths a more practical choice under tight compute budgets.

b Future Perspective

- Hyperparameter tuning has shown strong performance gains. Further improvements can be explored by tuning additional parameters such as numerical precision (e.g., 2/3/4 decimal places) and batch size (e.g., 4/8/12).
- Loss curves plateaued after 1500 steps in the final experiment, suggesting the model may have fully exploited the current dataset. Expanding the dataset could further enhance performance and generalisation.
- Currently, the model is trained using standard next-token prediction, where each predicted token is fed back as input. In contrast, teacher forcing provides the ground-truth token at each step during training. This method can be trialed and may help the model learn better temporal dependencies and improve training stability, especially for long sequences.
- In this study, LoRA is applied only to the query and value projection layers. Future experiments could explore applying LoRA to other components, such as the key projection, or test different combinations to identify the most effective placement.
- Adaptive LoRA[3] dynamically adjusts the rank of LoRA modules in each layer during training. Applying this method may improve efficiency by allocating capacity where it is most needed, which can reduce redundancy and boost performance.

8 Declaration

During the development of this project, I made use of Github Copilot's autocompletion feature to assist in coding and generating docstrings for functions. Additionally, ChatGPT was used to support debugging. For the final report, ChatGPT helped in generating LaTeX code in desired formats.

References

[1] Qwen Team. Qwen2.5 Technical Report, 2025. Accessed April 2025.

[2] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters, 2023.

[3] Haokun Zhang, Sijie Liu, Yankai Liu, Qiaolin Liu, Zicheng Lin, Hao Zhou, Chong Wang, and Peng Xu. Adaptive lora: Learnable rank adaption for low-rank adaptation of large language models. *arXiv preprint arXiv:2303.10512*, 2023.